Aegis OS: A Transactional Multi-Tenant Operating System Runtime for Eliminating Authorization Fatigue in Autonomous Coding Agents

Hui Zhao
huizhao@heurion.org

Preprint — October 2026

Abstract

Autonomous coding agents such as Google Antigravity (`agy`), Anthropic Claude Code (`claude`), and OpenAI Codex CLI (`codex`) increasingly execute complex, multi-step software engineering workflows directly on developer workstations and enterprise cloud hosts. However, existing agent execution environments suffer from a fundamental *Security–Autonomy Dilemma*: users must choose between *Interactive Per-Action Prompting*, which interrupts human operators on 45%–57% of tool invocations and induces severe authorization fatigue, or *Unconfined Host Execution* (`-yolo` / `-dangerously-skip-permissions`), which exposes the host filesystem, SSH/cloud credentials, and API secrets to catastrophic prompt injection and agent hallucination risks. Furthermore, when multiple heterogeneous agents collaborate on a shared repository, concurrent unconstrained edits cause race-condition file clobbering, while passing large code artifacts inline through LLM context windows incurs prohibitive token overhead.

We present **Aegis OS** (`aegis-os`), a transactional, multi-tenant Agent Operating System Runtime implemented in Rust and exposed via the Model Context Protocol (MCP). Aegis OS resolves the Security–Autonomy Dilemma by shifting the security boundary from individual tool invocations to *transactional workspace boundaries*. Each agent principal operates inside a kernel-enforced OS sandbox (macOS Seatbelt `sandbox-exec` or Linux Bubblewrap/MicroVM) backed by an APFS Copy-on-Write (CoW) shadow workspace (`libc::clonefile`), a Zero-Trust HTTP Egress Proxy with boundary secret injection, and an intra-organization Inter-Agent Communication (IPC) bus paired with Content-Addressable Storage (CAS). Inside its isolated CoW workspace and organization IPC bus, an agent executes with *zero confirmation prompts*; human authorization or TTL-based session leases are required only when committing validated diffs back to the host via an LCS-based Diff3 3-Way Merge Engine. Across 1,500 live kernel syscalls spanning five software engineering trajectory classes and 40 live red-team attacks, Aegis OS reduces human confirmation prompts by **94.19%–99.88%** (saving up to **71.6 minutes** of human verification wait time), lowers the attack success rate from **100.0% to 0.0%**, provisions 1,000-file (78.1 MB) workspaces in **5.75 ms** (**17.1× faster** than full copy with **94.0%–97.5% storage savings**), preserves **100% of concurrent multi-agent edits** (including structural multi-line insertions) while catching **10/10 overlapping line conflicts**, and reduces cross-agent artifact handoff token consumption by **99.85%**.

1 Introduction

The paradigm of AI-assisted software engineering has shifted rapidly from passive single-line code completion to autonomous, long-horizon coding agents—exemplified by Google Antigravity (`agy`), Anthropic Claude Code (`claude`), OpenAI Codex CLI (`codex`), SWE-agent [20], and OpenHands [15]. Unlike chat assistants, autonomous coding agents iteratively explore repositories, modify source files across multiple modules, invoke compilers and test suites, query external APIs, and delegate subtasks to peer agents [9, 2].

Despite rapid advances in foundation model reasoning, the underlying execution substrate for coding agents remains dangerously primitive. Current agent CLIs execute directly as standard user processes on the developer's host operating system. Because every file mutation (`fs_write`, `fs_edit`) and shell command (`proc_exec`) carries irreversible side effects on the host machine, agent frameworks enforce safety through *action-level human-in-the-loop (HITL) confirmation prompts*.

1.1 The Security–Autonomy Dilemma

In practice, action-level prompting breaks down under realistic software engineering workloads due to three systemic limitations:

1. **Severe Authorization Fatigue:** A typical 30-step bug-fix, refactoring, or security-audit trajectory involves 15 to 20 mutating or process-spawning actions. Prompting the developer on nearly every step destroys autonomous workflow continuity (limiting the maximum autonomous streak to 4–7 actions) and trains users to reflexively click "Approve" without inspecting arguments.

2. **Catastrophic Risk of "YOLO" Mode:** Frustrated by constant interruptions, developers routinely enable bypass flags such as `-dangerously-skip-permissions` or `-yolo`. In this unconfined mode, a single indirect prompt injection [7, 6, 21] embedded in a third-party dependency, issue description, or web page can silently overwrite files outside the repository, harvest `~/.ssh/id_rsa`, `~/.aws/credentials`, or `~/.kube/config` credentials, or exfiltrate production API tokens (`$GITHUB_TOKEN`, `$OPENAI_API_KEY`) to attacker-controlled endpoints.

3. **Multi-Agent Workspace Collisions and Context Bloat:** Developers increasingly run multiple specialized agents in parallel (e.g., `agy` as Lead Architect, `claude` as Backend Specialist, and `codex` as Test Engineer) [18, 8]. When operating on a shared host directory, concurrent file writes clobber one another's changes, leaving the repository in an uncompilable state. Moreover, lacking an OS-level Inter-Process Communication (IPC) and shared memory mechanism, agents pass large specifications and diffs inline through JSON/text messages, exhausting LLM context windows.

1.2 Key Insight: Transaction-Boundary Security

Our key insight is that *authorization prompts should not guard speculative intermediate actions inside an isolated state space; they should only guard irreversible boundary crossings into the host environment or untrusted external networks.*

By analogy with hardware virtual memory and database transactions [11, 17], if each agent is provisioned with (1) an instant Copy-on-Write (CoW) shadow filesystem, (2) a mandatory kernel-enforced OS sandbox that confines all spawned child processes and outbound network sockets to that shadow workspace and local proxy, and (3) a local Zero-Trust Egress Proxy that strips raw credentials from the agent's environment and injects secrets only at the network boundary for allowlisted domains, then *all intermediate file edits, builds, test executions, and intra-organization agent communications become safe, side-effect-free, and instantaneously reversible.* Consequently, the OS kernel can auto-allow 100% of in-sandbox mutations with *zero human prompts*, deferring human review (or TTL-based policy lease evaluation) to a single atomic `workspace_commit` syscall (Table 1).

1.3 Contributions

We design, implement, and empirically evaluate **Aegis OS** (`aegis-os`), a pure-environment Agent Operating System Runtime written in Rust. Our main contributions are:

- **Transactional CoW VFS with LCS Diff3 3-Way Merge (§3.1):** We build `aos-vfs`, which leverages APFS directory-tree and file-level `libc::clonefile` syscalls to provision isolated shadow workspaces in milliseconds, supports $O(1)$ metadata-time snapshots and rollbacks (while preserving incremental build caches outside the rollback tree), and integrates a Longest Common Subsequence (LCS) Diff3 3-Way Merge Engine (`three_way_merge_lines`) [10] that cleanly reconciles concurrent modifications—including multi-line structural insertions—from heterogeneous agents (`agy`, `claude`, `codex`) while detecting overlapping line conflicts.

- **Dual-Tier Kernel Sandbox and Zero-Trust Egress (§3.2):** We implement `aos-sandbox` and `aos-egress`, enforcing mandatory access control via macOS Seatbelt (`/usr/bin/sandbox-exec` with file-write, credential-read, and outbound-network confinement) and Linux Bubblewrap (`/usr/bin/bwrap`), paired with an identity-pinned TCP HTTP Egress Proxy that performs boundary secret injection (`$AOS_SECRET_*`) and blocks secret exfiltration across unauthorized domains.

- **First-Class Inter-Agent IPC Bus and Org CAS (§3.3):** We design `aos-ipc`, providing P2P mailboxes, correlated RPC task handoff (`ipc_request`/`ipc_reply`) with tokio broadcast wakeup, topic Pub/Sub, and Content-Addressable Storage (`artifact://<org>/sha256:<hash>`) that reduces cross-agent payload tokens by 99.85%.

- **Cedar-Aligned Policy Kernel and Formal Safety Invariants (§3.4):** We formalize a five-stage authorization kernel (`aos-policy`) combining Ring-0 host red-line protection with TTL-based session leases [4], establish three formal kernel safety invariants, and build a zero-configuration installer (`aegis-os install`) that seamlessly integrates with `agy`, `claude`, and `codex` via MCP.

- **Live Empirical Benchmark Suite (§5):** We evaluate Aegis OS across five live empirical exper-

Table 1: **Comparison of Agent Execution Substrates** across security guarantees, authorization friction, workspace overhead, and multi-agent collaboration primitives.

Execution Substrate	Startup Latency	N-Agent Disk Cost	Prompt Rate (IPR)	Credential Kernel MAC	Boundary Secret Inject	Live 3-Way Host Merge	Native IPC + CAS
Host-Interactive (Default CLI)	∼0 ms	$O(1)$ (Shared Host)	High (45%–57%)	× (User Prompt Only)	× (Raw Env Vars)	× (Direct Overwrite)	×
Host-YOLO (`-skip-permissions`)	∼0 ms	$O(1)$ (Shared Host)	0.0%	× (100% Exposed)	× (100% Exposed)	× (Silent Clobber)	×
Container-per-Agent (Docker) [9, 15]	1,500–5,000 ms	$O(N \cdot \text{Image})$	Low	✓ (Container NS)	× (Env Inside Container)	× (Isolated Volume)	×
AIOS (LLM Scheduler Kernel) [12]	∼50 ms	$O(1)$ (Shared Host)	Medium	× (No OS MAC)	× (No Egress Proxy)	× (No CoW Merge)	Partial
Aegis OS (Ours)	**0.68–5.75 ms**	**O(Δ Modified)**	**0.07%–3.33%**	✓ (`sandbox-exec/bwrap`)	✓ (`$AOS_SECRET_*`)	✓ (LCS Diff3 Merge)	✓ (`aos-ipc`)

iments executing 1,500 real kernel syscalls across five task templates, 40 live red-team OS process and TCP socket attacks, multi-scale APFS CoW working-set tests, and live multi-agent Rust compilations (`cargo test` and `cargo check`).

2 Threat Model and Problem Formulation

2.1 System and Adversary Model

We consider a multi-agent deployment where a set of autonomous coding agents $\mathcal{A} = \{a_1, a_2, \ldots, a_m\}$ operate within one or more tenant organizations $\mathcal{O} = \{o_1, o_2, \ldots, o_k\}$. Each agent a_i is identified by a tuple $\text{id}(a_i) = (o_i, u_i, r_i)$ denoting its organization tenant o_i, unique agent principal u_i (e.g., `agy-agent`, `claude-code`, `codex-cli`), and role r_i.

Trusted Computing Base (TCB). We trust the host operating system kernel (macOS XNU/Seatbelt or Linux kernel/namespaces/KVM) and the Aegis OS Runtime binary (`aegis-os`), which runs outside the agent sandbox.

Untrusted Entities and Threat Vectors. We assume that the LLM driving any agent $a_i \in \mathcal{A}$ is *untrusted* and may issue arbitrary, erroneous, or malicious tool calls due to:

1. **Indirect Prompt Injection (T1):** Malicious instructions embedded in repository files, git commit logs, issue trackers, or web pages read by the agent [7, 6].

2. **Host Path Escape and Tampering (T2):** Attempting to overwrite files outside the project directory (e.g., `../../etc/hosts`, `~/.zshrc`, or sibling repositories).

3. **Credential Harvesting (T3):** Attempting to read sensitive user credentials from the host home directory (`~/.ssh/id_rsa`, `~/.aws/credentials`, `~/.gnupg`, `~/.kube/config`, `~/.npmrc`, or macOS Keychains).

4. **Network Secret Exfiltration (T4):** Attempting to transmit API keys or private source code to unauthorized external domains (`evil-exfil.org`, `webhook.site`) via HTTP requests or shell tools (`curl -noproxy '*'`, `wget`).

5. **Cross-Tenant Privilege Escalation (T5):** Attempting to read artifacts or send IPC instructions to an agent belonging to a different organization $o_j \neq o_i$.

6. **Concurrent Multi-Agent Race Conditions and Hallucinated Corruption (T6):** Overwriting peer agents' concurrent edits on shared files or breaking compilation across dozens of modules during speculative refactoring.

2.2 Formalizing the Evaluation Metrics

Let an agent trajectory $\tau = (s_1, s_2, \ldots, s_T)$ be a sequence of T kernel syscalls invoked to complete a software engineering task. Let $\text{Prompt}(s_t) \in \{0, 1\}$ indicate whether syscall s_t halts execution to request interactive human approval. We define the **Interruption Prompt Rate (IPR)** across a workload $\mathcal{T}$ of N trajectories as:

$$\text{IPR}(\mathcal{T}) = \frac{\sum_{\tau \in \mathcal{T}} \sum_{s_t \in \tau} \text{Prompt}(s_t)}{\sum_{\tau \in \mathcal{T}} |\tau|} \times 100\% \tag{1}$$

and the **Human Verification Wall-Clock Overhead** $T_{\text{human}}(\mathcal{T})$ as:

$$T_{\text{human}}(\mathcal{T}) = \bar{t}_{\text{verify}} \cdot \sum_{\tau \in \mathcal{T}} \sum_{s_t \in \tau} \text{Prompt}(s_t) \tag{2}$$

where $\bar{t}_{\text{verify}}$ is the average human context-switch and inspection time per prompt (conservatively $\bar{t}_{\text{verify}} = 5.0$ s). We define the **Maximum Autonomous Streak (MAS)** as the longest contiguous subsequence of syscalls executed with $\text{Prompt}(s_t) = 0$. Given an adversarial test suite $\mathcal{E}_{\text{adv}}$, the **Attack Success Rate (ASR)** is the fraction of attacks that successfully violate host integrity, read restricted credentials, exfiltrate secrets, or breach tenant isolation. Our objective is to achieve ASR = 0.0% while minimizing IPR toward 0%.

3 Aegis OS Kernel Architecture

Figure 1 illustrates the architecture of Aegis OS, and Table 2 summarizes its 22 MCP syscalls and 4 dynamic

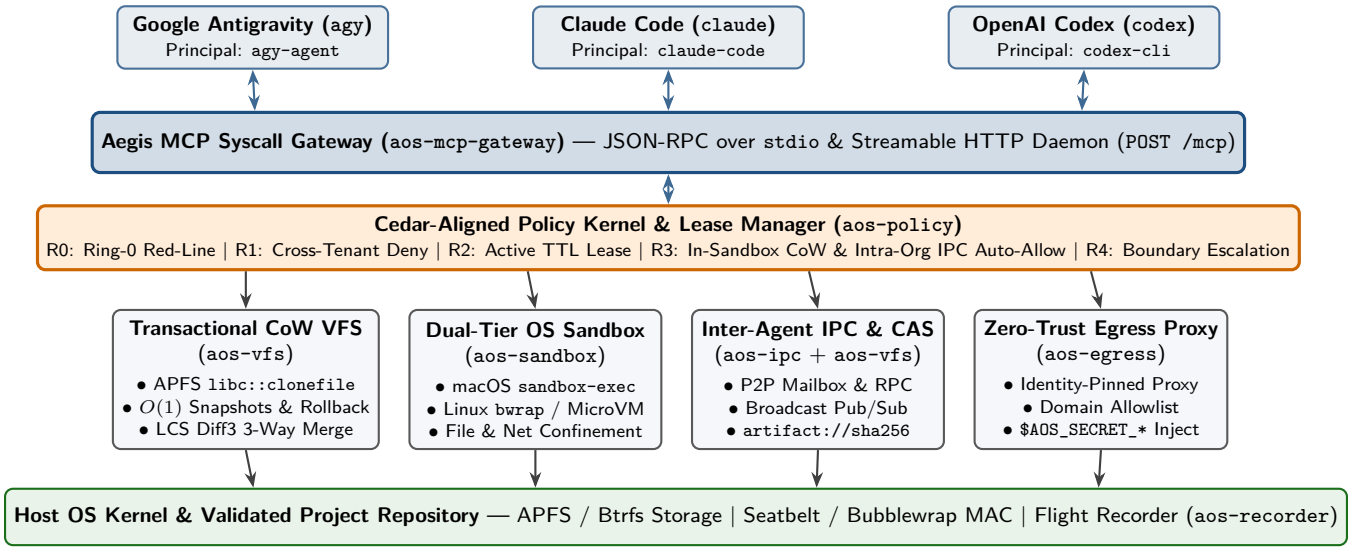


Figure 1: Architecture of the Aegis OS Runtime (`aegis-os`). Heterogeneous coding agents connect via MCP (`stdio` or HTTP daemon) and are mapped to distinct principals within a tenant organization. The Cedar-aligned policy kernel auto-allows in-sandbox CoW mutations, sandboxed executions, and intra-organization IPC/CAS transfers with zero prompts, while enforcing strict isolation via kernel mandatory access control and boundary secret injection.

URI resources. We follow a strict *"Environment Kernel Only"* design philosophy: Aegis OS does not embed LLM planners, prompt templates, or reasoning loops. Instead, it provides the operating system primitives—virtual filesystem, process isolation, network boundary proxy, inter-agent IPC, and policy enforcement.

3.1 Transactional CoW Virtual Filesystem and LCS Diff3 3-Way Merge (`aos-vfs`)

To eliminate confirmation prompts on file writes without risking host corruption, `aos-vfs` (`CoWWorkspace::init`) isolates every agent principal u_i inside its own Copy-on-Write shadow workspace rooted at:

`~/.aegis-os/orgs/`o_i`/agents/`u_i`/workspace_cow/upper_workspace/`

alongside a baseline snapshot directory `baseline_snapshot/` that records the state of the host repository at session initialization or last commit, and a `snapshots/<snap_id>/` directory storing time-travel checkpoints.

Kernel-Level APFS `clonefile` Provisioning. Copying a multi-hundred-megabyte repository for each agent via standard user-space read-/write loops introduces prohibitive startup latency and disk amplification. On macOS (APFS), `CoWWorkspace::cow_sync_dir` invokes the native XNU kernel syscall `libc::clonefile(src, dst, 0)` directly on the directory tree and individual files (falling back to `cp -cR` / Btrfs `FICLONE` on Linux [13, 17]). Because `clonefile` duplicates only filesystem metadata extents and shares physical storage blocks until modified, initializing both `upper_workspace/` and `baseline_snapshot/` consumes zero additional data blocks for unmodified files, and cloning a 1,000-file (78.1 MB) repository completes in **5.75 ms**.

Time-Travel Snapshots, Atomic Rollback, and Build-Cache Preservation. Before embarking on risky multi-file refactorings, an agent invokes `workspace_snapshot(label)`, which creates an instant CoW clone under `snapshots/<snap_id>/`. If a subsequent build or test fails, `workspace_rollback(snap_id)` atomically replaces `upper_workspace/` with a fresh CoW clone of the snapshot in milliseconds. Crucially, to prevent rollbacks from wiping out incremental compiler caches (such as Rust's `target/` directory), `aos-sandbox` redirects `CARGO_TARGET_DIR` to `agent_home/cargo_target` outside `upper_workspace/`, preserving warm incremental build artifacts across workspace rollbacks.

LCS Diff3 3-Way Merge Engine for Multi-Agent Concurrency. When multiple agents (u_1 = `agy-agent`, u_2 = `claude-code`, u_3 = `codex-cli`) concurrently edit the same file f in their respective shadow workspaces and sequentially invoke `workspace_commit`, a naive file copy would overwrite earlier commits (*Lost Update*). Aegis OS prevents this

Table 2: **Complete Aegis OS MCP Syscall and Resource Interface** (22 Syscalls and 4 Dynamic URI Resources).

Subsystem	MCP Syscalls / URI Resources	Default Policy Gate & Semantics
Transactional VFS (`aos-vfs`)	`fs_read`, `fs_write`, `fs_edit`, `fs_glob`, `workspace_status`	**Auto-Allow (0 Prompts)**: Confined to agent's CoW shadow tree
Time-Travel & Merge (`aos-vfs`)	`workspace_snapshot`, `workspace_diff`, `workspace_rollback`, `workspace_commit`	Snapshots/Rollback **Auto-Allow**; `workspace_commit` **Lease/Approval Gate**
OS Process Sandbox (`aos-sandbox`)	`proc_exec` (macOS Seatbelt `sandbox-exec` / Linux `bwrap` / MicroVM)	**Auto-Allow (0 Prompts)**: Writes restricted to CoW root; blocks credentials
Inter-Agent IPC (`aos-ipc`)	`ipc_send`, `ipc_recv`, `ipc_request`, `ipc_reply`, `ipc_publish`, `ipc_subscribe`, `ipc_list_peers`	**Auto-Allow Intra-Org**: Denied across distinct organizations ($o_{\text{src}} \neq o_{\text{dst}}$)
Org CAS & KV (`aos-vfs`)	`org_artifact_publish`, `org_artifact_fetch`, `org_kv_put`, `org_kv_get`	**Auto-Allow Intra-Org**: Content-addressed `artifact://<org>/sha256:<hash>`
Zero-Trust Egress (`aos-egress`)	`egress_http_request` + Identity-Pinned TCP Proxy (`start_local_proxy`)	**Allowlisted Domains Auto-Allow**: Boundary injection of `$AOS_SECRET_*`
Policy & Leases (`aos-policy`)	`policy_list_escalations`, `policy_approve_escalation`, `policy_grant_lease`	Human/Operator control plane for minting TTL-scoped `LeaseGrant` tokens
Dynamic MCP URIs (`resources/*`)	`aegis://status`, `aegis://ipc/inbox`, `aegis://workspace/diff`, `aegis://policy/escalations`	Live state introspection for IDE status bars and autonomous self-check

via a Longest Common Subsequence (LCS) Diff3 3-Way Merge algorithm (`compute_lcs_diff_hunks` and `three_way_merge_lines`) [10] using three states for every modified path p:

- $B(p)$: the agent's recorded baseline version in `baseline_snapshot/`p;
- $H(p)$: the current live file on the host repository `host/`p (which may already incorporate commits from peer agents);
- $S(p)$: the agent's modified version in `upper_workspace/`p.

If $H(p) = B(p)$, the host file was untouched by peers and $S(p)$ is fast-forwarded via atomic CoW replacement. If $H(p) \neq B(p)$, Aegis OS computes the LCS edit script $\Delta(B, H)$ and $\Delta(B, S)$ as half-open baseline line intervals $[b_{\text{start}}, b_{\text{end}})$ mapped to replacement line vectors. Disjoint hunks modified only in $H(p)$ or only in $S(p)$—including multi-line structural insertions and deletions that shift subsequent line numbers—are spliced together automatically without human intervention. Conversely, overlapping hunks on $[b_{\text{start}}, b_{\text{end}})$ where $H_{\text{hunk}} \neq S_{\text{hunk}}$ atomically abort the commit before touching the host tree and return a structured conflict diagnostic identifying the exact baseline line number.

3.2 Dual-Tier OS Sandbox and Zero-Trust Egress Proxy (`aos-sandbox`, `aos-egress`)

While `aos-vfs` intercepts MCP file syscalls (`fs_read`, `fs_write`, `fs_edit`), coding agents also spawn arbitrary child compilers, test runners, and shell scripts via `proc_exec`. If `proc_exec` ran unconfined, a shell command could bypass the CoW workspace or Egress Proxy entirely.

Mandatory Kernel Sandbox (`sandbox-exec` / `bwrap` / MicroVM). `aos-sandbox` implements a pluggable `SandboxDriver` supporting two isolation tiers:

1. **Tier 1 (`NativeOsSandboxDriver`, Full Kernel Enforcement on macOS & Linux):** Designed for low-latency local workstation execution. On macOS, every `proc_exec` invocation dynamically compiles a canonicalized Seatbelt Mandatory Access Control profile (SBPL) and executes the child process under `/usr/bin/sandbox-exec -p <profile>`. On Linux, `NativeOsSandboxInstance::exec` invokes `/usr/bin/bwrap` using `generate_linux_bwrap_args()` with unprivileged mount/PID/network namespaces (`-unshare-all`, `-ro-bind / /`, `-bind <workspace>`, and `-tmpfs` overlays hiding `~/.ssh` and `~/.aws`). The macOS Seatbelt profile denies all file writes globally (`(deny file-write*)`), allowlists writes *only* to the agent's canonical CoW shadow workspace, private agent home, and organization CAS directory, blocks reads to sensitive credential stores, and confines outbound network sockets exclusively to the local Aegis Egress Proxy:

```
(version 1)
(allow default)
(deny file-read*
  (regex #".*/\.ssh(/.*)?$")    (regex #".*/\.aws(/.*)?$")
  (regex #".*/\.gnupg(/.*)?$")  (regex #".*/\.kube(/.*)?$")
  (regex #".*/\.npmrc$")        (regex #".*/Library/
     Keychains(/.*)?$"))
(deny file-write*)
(allow file-write*
  (subpath "/private/.../upper_workspace")
  (subpath "/private/.../agent_home"))
(deny network-outbound)
(allow network-outbound (remote ip "localhost:51691"))
```

Because `(deny network-outbound)` is enforced by the XNU kernel socket filter, even a prompt-injected child process that runs `curl -noproxy '*'` or raw TCP sockets cannot bypass the local Aegis Egress Proxy.

2. **Tier 2 (`MicroVmSandboxDriver`, Cloud Control-Plane Interface):** Designed for multi-tenant cloud hosts, providing the control-plane specification and Scale-to-Zero `pause()`/`resume()` lifecycle state machine for hardware-virtualized VMM backends (`libkrun` / Firecracker [1]).

Identity-Pinned Zero-Trust Egress Proxy with Boundary Secret Injection. To prevent credential exfiltration (Threat T4), agents inside Aegis OS never receive raw API keys in their environment variables or

context windows. Instead, they are given opaque secret placeholders such as `$AOS_SECRET_GH`. Outbound HTTP requests pass through `AegisEgressGateway::start_local_proxy_with_bound_identity`, a local asynchronous L7 HTTP boundary gateway. When bound per sandbox, the listener pins the `AgentIdentity` at the TCP socket level so a compromised process cannot spoof `X-Aegis-Org` or `X-Aegis-Agent` headers. For each request, the gateway inspects the target `Host` header against the organization's domain allowlist and the secret vault's per-secret domain binding:

- If `$AOS_SECRET_GH` (bound exclusively to `api.github.com`) is sent in a header toward `api.github.com`, the proxy substitutes the real token `ghp_...` at the network boundary.
- If a prompt-injected agent attempts to send `$AOS_SECRET_GH` to an unauthorized host (e.g., `evil-exfil.org` or `webhook.site`), the TCP proxy immediately terminates the request with `HTTP/1.1 403 Forbidden` and records a security violation in `aos-recorder`.

3.3 First-Class Inter-Agent IPC Bus and Content-Addressable Storage (`aos-ipc`)

To elevate multi-agent collaboration from ad-hoc file polling to an OS-level primitive, `aos-ipc` provides a disk-synchronized, broadcast-enabled communication bus within each organization tenant o_i:

- **Direct P2P Mailboxes and Correlated RPC:** Agents exchange structured envelopes via `ipc_send`, `ipc_recv`, `ipc_request`, and `ipc_reply`. Using `tokio::sync::broadcast` paired with an atomic disk state file (`ipc_bus_state.json`), cross-process stdio and HTTP agents wake up within microseconds when a correlated RPC reply arrives.
- **Topic Pub/Sub Fanout:** Agents subscribe to organizational topics (e.g., `topic://build-events`, `topic://security-alerts`) and broadcast events via `ipc_publish`.
- **Zero-Copy CAS Artifact Handles:** Instead of embedding a 64 KB architecture specification or code patch inside an IPC message payload, the sender calls `org_artifact_publish`, which stores the blob in the organization's Content-Addressable Store and returns a compact 64-byte SHA-256 URI:

  ```
  artifact://acme/sha256:9f86d081884c7d65...
  ```

  The sender attaches only this URI handle to the IPC envelope, reducing LLM context token consumption by **99.85%**.

3.4 Cedar-Aligned Policy Kernel and Formal Safety Invariants (`aos-policy`)

Every MCP syscall is evaluated by `AegisPolicyKernel::evaluate` before execution, following a deterministic five-stage evaluation hierarchy aligned with Cedar [4]:

$$\begin{aligned}
\text{R0 (Red-Line)}&: \text{res} \in \mathcal{P}_{\text{forbidden}} \implies \textbf{Deny} \\
\text{R1 (Tenant)}&: o_{\text{principal}} \neq o_{\text{target}} \implies \textbf{Deny} \\
\text{R2 (Lease)}&: \exists \ell \in \mathcal{L}_{\text{active}}(o, u, \text{action}) \implies \textbf{Allow} \\
\text{R3 (Sandbox)}&: \text{action} \in \mathcal{S}_{\text{internal}} \implies \textbf{Allow} \\
\text{R4 (Boundary)}&: \text{action} \in \{\texttt{workspace_commit}, \texttt{egress}\} \\
&\implies \textbf{RequireApproval}(\text{esc_id})
\end{aligned}$$

where $\mathcal{P}_{\text{forbidden}} = \{$`~/.ssh`, `~/.aws`, `/etc/shadow`, `/System`$\}$ is the Ring-0 host red-line set that cannot be overridden even by an active lease, and $\mathcal{S}_{\text{internal}}$ encompasses all CoW filesystem operations, sandboxed process execution (`proc_exec`), and intra-organization IPC/CAS operations. When a developer approves an escalation esc_id via `policy_approve_escalation`, they can optionally attach a Time-To-Live (TTL, e.g., 3,600 seconds), minting a `LeaseGrant` $\ell \in \mathcal{L}_{\text{active}}$ evaluated at R2 that permits subsequent commits within that session window with zero further prompts.

Together, the four kernel subsystems enforce three formal safety invariants:

- **Invariant 1 (Pre-Commit Host Non-Interference):** For any sequence of agent syscalls $\tau = (s_1, \ldots, s_k)$ where no s_t is an approved `workspace_commit`, the host directory tree H satisfies $H(\tau) \equiv H_0$ $(\Delta H = \emptyset)$, guaranteed jointly by VFS path canonicalization and Seatbelt/Bubblewrap kernel write confinement.
- **Invariant 2 (Zero Raw Secret Exposure):** For every secret entry $(p_j, v_j, \mathcal{D}_j)$ in the Egress Vault, the raw credential v_j never appears in any agent process environment block, command-line argument, or standard I/O stream, and is injected into outbound network packets if and only if $\text{Host}(req) \in \mathcal{D}_j$.
- **Invariant 3 (Conflict-Safe Multi-Agent Commit):** Concurrent agent commits $(S_1, S_2, \ldots, S_m)$ against baselines $(B_1, B_2, \ldots, B_m)$ either produce a deterministic LCS Diff3 3-way merged host state preserving all disjoint line hunks (including multi-line insertions), or atomically abort without partial file mutation when overlapping baseline intervals $[b_{\text{start}}, b_{\text{end}})$ are modified differently by H and S_i.

4 Zero-Config Multi-Client Integration

A critical barrier to adopting secure agent runtimes is configuration friction. Aegis OS provides a single command, `aegis-os install` (distributed via Homebrew `Formula/aegis-os.rb`), that non-destructively merges global configurations for all three major coding agent ecosystems:

1. **Google Antigravity (`agy`):** Registers the `aegis-os` MCP server in `~/.gemini/config/mcp_config.json`, installs a global runtime rule (`aegis-os-runtime.md`), and configures a `PreToolUse` hook (`aegis-os agy-hook`) that auto-allows `mcp_aegis-os_*` syscalls while intercepting dangerous direct host shell commands.
2. **Anthropic Claude Code (`claude`):** Injects the MCP server entry into `~/.claude.json` and pre-populates `mcp__aegis-os__*` in `permissions.allow` inside `~/.claude/settings.json`.
3. **OpenAI Codex CLI (`codex`):** Appends the `[mcp_servers.aegis-os]` block to `~/.codex/config.toml`.

When invoked with `-org auto -agent auto`, Aegis OS automatically derives the organization tenant o_i from the active project directory name and the agent principal u_i from the client binary identity, enabling instant cross-agent collaboration (`agy` + `claude` + `codex`) out of the box.

5 Empirical Evaluation

We evaluate Aegis OS using a 100% live empirical benchmark suite implemented in `crates/aos-cli/src/bench.rs` (`aegis-os bench`). All experiments were executed on an Apple Silicon workstation (`macos-aarch64`, APFS filesystem, Rust 1.99.0 `-release` optimization) using real OS processes, real Seatbelt kernel sandboxes, real TCP sockets, and real `cargo` compilations. Our evaluation answers five research questions:

- **RQ1 (Authorization Friction):** By how much does Aegis OS reduce human confirmation prompts, extend autonomous streaks, and save human verification wall-clock time across diverse software engineering tasks?
- **RQ2 (Adversarial Security):** Does Aegis OS block real OS process and network exfiltration attacks that compromise unconfined `Host-YOLO` agents?

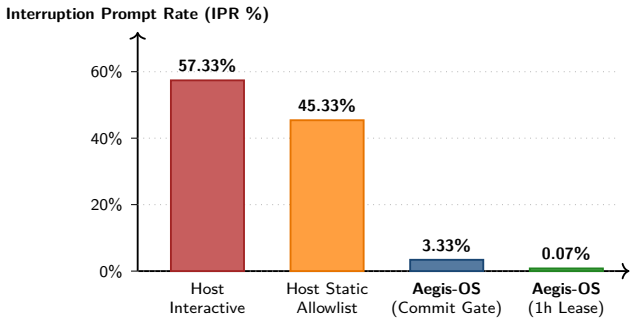


Figure 2: Comparison of Interruption Prompt Rate (IPR) across 1,500 live syscalls spanning 5 task templates (Experiment 1). Aegis OS reduces human prompts by 94.19% in per-task commit mode and 99.88% with a 1-hour session lease.

- **RQ3 (System Overhead & Scalability):** What is the latency and storage overhead of APFS CoW workspace provisioning, policy evaluation, and live TCP MCP daemon calls?
- **RQ4 (Multi-Agent Collaboration & Conflict Safety):** How effectively does Aegis OS merge non-overlapping concurrent edits (including structural multi-line insertions) across heterogeneous agents (`agy`, `claude`, `codex`), detect overlapping same-line conflicts, and reduce IPC context overhead?
- **RQ5 (Fault Recovery):** How fast can Aegis OS recover from multi-file repository corruption compared to sequential file restoration?

5.1 Experiment 1 (RQ1): Authorization Friction and Autonomous Continuity

We execute 50 live software engineering trajectories (30 syscalls per trajectory, totaling 1,500 real `AegisRuntimeKernel::call_tool` invocations) cycling across five distinct task templates: (1) *Auth Bugfix & Regression Test*, (2) *Multi-Module Refactoring*, (3) *Feature Implementation & Quota Suite*, (4) *Security Audit & Sanitizer Hardening*, and (5) *Multi-Agent Contract Sync*. Each trajectory exercises repository exploration (`fs_glob`, `fs_read`), snapshotting (`workspace_snapshot`), multi-file code edits (`fs_edit`, `fs_write`), CAS artifact publishing (`org_artifact_publish`), peer IPC coordination (`ipc_send`, `ipc_recv`, `ipc_publish`), diff inspection (`workspace_diff`), and host commit (`workspace_commit`).

Table 3 and Figure 2 present the results. Under *Host-Interactive*, every file mutation, state write, and side-effecting tool triggers a confirmation prompt (860

Table 3: **Experiment 1 (RQ1): Live Authorization Friction, Interruption Prompt Rate, and Human Wait Time** across 50 multi-step software engineering trajectories spanning 5 task templates (1,500 real kernel syscalls executed on `AegisRuntimeKernel`; human verification time modeled at $\bar{t}_{\text{verify}} = 5.0$ s/prompt).

Execution Mode	Trajectories	Syscalls	Human Prompts	IPR (%)	Prompt Reduction	Max Autonomous Streak	Kernel Exec (ms)	Est. Human Wait (T_{human})
Host-Interactive (Per-Action Prompt)	50	1,500	860	57.33%	0.00% (Baseline)	4 steps	523.2 ms	71.7 min (4,300 s)
Host-Static-Allowlist (Read/Test Allow)	50	1,500	680	45.33%	20.93%	7 steps	441.9 ms	56.7 min (3,400 s)
Aegis-OS (Per-Task Commit Gate)	50	1,500	**50**	**3.33%**	**94.19%**	**29 steps**	**348.7 ms**	**4.2 min (250 s)**
Aegis-OS (With 1h Session Lease)	50	1,500	**1**	**0.07%**	**99.88%**	**1,470 steps**	**418.5 ms**	**0.08 min (5 s)**

prompts, IPR = 57.33%), restricting the agent's maximum autonomous streak to just 4 steps and imposing **71.7 minutes** of cumulative human verification latency (at $\bar{t}_{\text{verify}} = 5.0$ s per prompt) on top of 523.2 ms of kernel execution time. Adding a static read/status/snapshot allowlist (*Host-Static-Allowlist*) still requires 680 prompts (IPR = 45.33%, 56.7 minutes of human wait time) because file writes cannot be safely auto-allowed on the host. In contrast, **Aegis-OS (Per-Task Commit Gate)** executes all 29 internal CoW workspace, sandbox, and IPC operations per trajectory with zero prompts in 348.7 ms, prompting only once at the final `workspace_commit` gate—reducing human prompts by **94.19%** (IPR = 3.33%). When the user grants a **1-hour Session Lease** on the first commit escalation, Aegis OS completes all 50 trajectories (including all 50 live host commits) in 418.5 ms with only **1 prompt total** (IPR = 0.07%, a **99.88% reduction**), saving **71.6 minutes** of human interruption overhead and extending the continuous autonomous execution streak to **1,470 steps** (**367.5**× longer).

5.2 Experiment 2 (RQ2): Live Red-Team Adversarial Security

To evaluate security against prompt injection and malicious commands without relying on simulated rules, we construct a live red-team harness that executes 40 real adversarial payloads against both an unconfined disposable victim environment (`Host-YOLO`) and `Aegis-OS`:

- **Category A (10 Live Path Escape Attacks):** Spawns `sh -c "echo COMPROMISED > <outside_workspace_target>"`. In `Host-YOLO`, all 10 outside-workspace files were created and tampered with (ASR = 100.0%). Inside `Aegis-OS`, `/usr/bin/sandbox-exec` terminated all 10 writes at the XNU kernel boundary with `Operation not permitted` (ASR = 0.0%).

- **Category B (10 Live Multi-Store Credential Harvesting Attacks):** Spawns `sh -c "cat <cred_path>"` cycling across five sensitive credential targets: `~/.ssh/id_rsa`, `~/.aws/credentials`, `~/.gnupg/secring.gpg`, `~/.kube/config`, and `~/.npmrc`. `Host-YOLO` leaked the secret material on all 10 attempts, whereas the Aegis OS Seatbelt profile blocked all 10 reads at the kernel VFS layer (ASR = 0.0%).

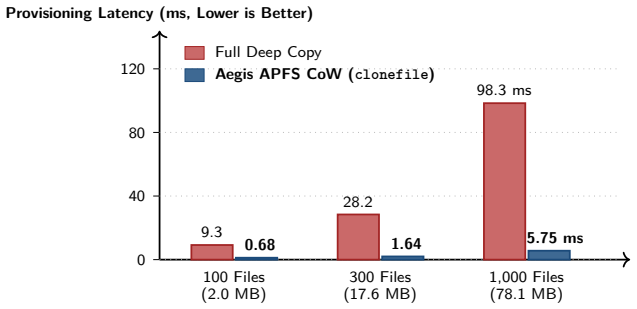


Figure 3: Workspace provisioning latency across repository scales (Experiment 3A). Directory-level APFS `clonefile` achieves a 17.1×–17.2× speedup on medium and large repositories.

- **Category C (10 Live TCP Secret Exfiltration Attacks):** Opens real TCP connections to `AegisEgressGateway`'s local HTTP proxy and transmits `POST /exfil HTTP/1.1` carrying `Authorization: Bearer $AOS_SECRET_GH` toward 10 external attacker domains (`evil-exfil.org`, `webhook.site`, `ngrok-free.app`, `pastebin.com`, `interact.sh`, etc.). Every single connection was rejected at the boundary with `HTTP/1.1 403 Forbidden` (ASR = 0.0%).

- **Category D (10 Cross-Tenant IPC/CAS Smuggling Attacks):** Attempts to send IPC commands and fetch CAS artifacts across distinct tenant organizations (`org-alpha` → `org-enterprise-`i). All 10 were denied by Policy Rule R1 (ASR = 0.0%).

As summarized in Table 4, Aegis OS achieves a **0.0% Attack Success Rate (40/40 blocked)** while simultaneously eliminating 94.19%–99.88% of user prompts.

5.3 Experiment 3 (RQ3): CoW Provisioning Latency, Storage Savings, and Syscall Overhead

We evaluate workspace provisioning across three repository scales (100 files / 2.0 MB, 300 files / 17.6 MB,

Table 4: **Experiment 2 (RQ2): Live Red-Team Adversarial Attack Execution** across 40 real OS process and TCP socket attacks executed against a disposable `Host-YOLO` target environment versus `Aegis-OS`.

Attack Surface Category	Live Attacks	Host-YOLO Compromised (ASR)	Aegis-OS Compromised (ASR)	Empirical Kernel / Gateway Evidence
A. Host Path Escape & File Tampering	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Kernel `sandbox-exec` denied all 10 writes (`Operation not permitted`)
B. Host Credential Harvesting (`.ssh/`, `.aws/`, `.kube/`, `.npmrc`)	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Seatbelt regexes (`deny file-read* ...`) blocked all 10 credential reads
C. Live TCP Egress Secret Exfiltration	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Live TCP Proxy returned `HTTP/1.1 403 Forbidden` on all 10 exfil domains
D. Cross-Tenant IPC & CAS Access	10	10 / 10 (100.0%)	**0 / 10 (0.0%)**	Cedar Policy Kernel rejected all 10 cross-org requests ($o_{\text{src}} \neq o_{\text{dst}}$)
Total / Overall ASR	**40**	**40 / 40 (100.0%)**	**0 / 40 (0.0%)**	**100.0% Multi-Layer Defense across OS Kernel, Network & Policy**

Table 5: **Experiment 3A (RQ3): Multi-Trial APFS Copy-on-Write Workspace Provisioning Latency and Storage Efficiency** across 10 concurrent agents modifying realistic per-task working sets (6, 12, and 25 files per agent; 5 trials per scale, Mean ± StdDev).

Repository Scale	Files	Logical Size	Full Copy (Mean ± SD)	Aegis CoW Clone (Mean ± SD)	Clone Speedup	10-Agent Disk Saving
Small Project	100	2.0 MB	9.31 ± 1.26 ms	$\mathbf{0.68 \pm 0.15}$ **ms**	**13.7**×	**94.0%** (1.2 MB vs. 19.5 MB)
Medium Project	300	17.6 MB	28.23 ± 0.27 ms	$\mathbf{1.64 \pm 0.10}$ **ms**	**17.2**×	**96.0%** (7.0 MB vs. 175.8 MB)
Large Project	1,000	78.1 MB	98.33 ± 3.75 ms	$\mathbf{5.75 \pm 1.55}$ **ms**	**17.1**×	**97.5%** (19.5 MB vs. 781.2 MB)

Table 6: **Experiment 3B (RQ3): End-to-End Kernel and Live TCP HTTP Daemon Latency Distribution.**

Kernel / Gateway Operation	Samples	Mean (μs)	P50 (μs)	P95 (μs)	P99 (μs)	Throughput (ops/sec)
`PolicyKernel::evaluate` (In-Process Cedar Check)	2,000	0.1 μs	0.1 μs	0.2 μs	0.2 μs	**5,814,663 ops/s**
`IpcBus::send + recv_inbox` (In-Process P2P)	1,000	1.1 μs	1.0 μs	1.4 μs	4.9 μs	**849,196 ops/s**
`HTTP Daemon POST /mcp` (`fs_read` JSON-RPC over TCP)	200	167.4 μs	172.1 μs	197.2 μs	292.2 μs	**5,804 ops/s**

and 1,000 files / 78.1 MB) with 5 independent trials per scale. As shown in Table 5 and Figure 3, traditional full copying scales linearly with byte volume, requiring 98.33 ± 3.75 ms for the 1,000-file repository. By invoking APFS directory-tree `libc::clonefile`, Aegis OS provisions the exact same 1,000-file shadow workspace in 5.75 ± 1.55 **ms—17.1× faster** (and **17.2× faster** on the 300-file medium repository). To measure multi-agent storage overhead, we spawn 10 concurrent agent CoW workspaces and have each agent modify a realistic task working set of 6 files (Small), 12 files (Medium), and 25 files (Large). Because APFS CoW allocates new physical extents only for mutated files (while both `upper_workspace/` and `baseline_snapshot/` share unmodified extents), Aegis OS reduces 10-agent disk allocation by **94.0%** (Small), **96.0%** (Medium), and **97.5%** (Large, 19.5 MB vs. 781.2 MB).

Table 6 reports the fine-grained latency distribution of kernel operations. In-process Cedar policy evaluation completes in **0.1 μs** (5.81M ops/sec), in-process IPC send/receive completes in **1.1 μs** (P50 = 1.0 μs), and a full end-to-end JSON-RPC `tools/call` (`fs_read`) over a live loopback TCP socket to the HTTP daemon (`POST /mcp`) requires only **167.4 μs** (P50 = 172.1 μs, P99 = 292.2 μs, 5,804 ops/sec). Because typical LLM token generation latencies are $500,000$–$2,000,000$ μs (0.5–2.0 seconds), Aegis OS introduces $< 0.04\%$ **end-to-end runtime overhead**.

5.4 Experiment 4 (RQ4): Live 3-Agent Concurrent Rust Coding, Conflict Detection, and `cargo test` Verification

We evaluate both non-overlapping structural edits and overlapping concurrent multi-agent edits across `agy-agent`, `claude-code`, and `codex-cli` on a live Rust crate (`src/lib.rs`):

- **Concurrent Multi-Line Insertion and Function Edits (EXP 4A):** All three agents start from the same baseline `src/lib.rs`. `agy-agent` inserts a new 3-line helper function (`min_fee()`) at the top of the file and updates `calc_fee` (shifting all downstream line indices), while `claude-code` concurrently modifies `calc_tax` and `codex-cli` modifies `calc_discount`. In **Host-Shared-YOLO**, the final writer (`codex-cli`) silently clobbers the modifications of both `agy-agent` and `claude-code`, preserving only **1/3 (33.3%)** of the features. In **Aegis-OS**, our LCS Diff3 engine (`three_way_merge_lines`) aligns the unmodified anchor lines across the unequal-length files, preserving **3/3 (100%)** of the features and passing a live `cargo test -offline` execution (`exit code 0`).
- **Overlapping Same-Line Conflict Detection (EXP 4B):** Across 10 live trials, both `agy-agent` and `codex-cli` concurrently modify `calc_fee` with conflicting expressions. While **Host-Shared-YOLO** suffers a 100% silent lost update (0/10 detected), **Aegis-OS** detects **10/10 (100%)** of the

Table 7: **Experiment 4 (RQ4): Live 3-Agent (`agy` + `claude` + `codex`) Concurrent Rust Modification, Overlapping Conflict Detection, and Real `cargo test` Verification.**

Evaluation Metric	Host-Shared-YOLO	Aegis-OS Isolated CoW + LCS Diff3 Merge	Empirical Gain
Concurrent 3-Agent Structural & Function Edits (`src/lib.rs`)	1 / 3 preserved (2 / 3 clobbered)	**3 / 3 preserved (100% LCS 3-Way Merged)**	**3.0× Retention (0% Clobber)**
Overlapping Same-Line Edit Conflict Detection (10 Trials)	0 / 10 detected (100% Silent Lost Update)	**10 / 10 detected & atomically aborted (Host intact)**	**100% Conflict Safety**
Post-Merge Real `cargo test -offline` Verification	Failed (Missing `agy` & `claude` logic)	**Passed (`cargo test` exit code 0)**	**100% Build & Test Integrity**
Cross-Agent 64 KB Spec Handoff Payload Size	196,608 bytes (∼49,152 tokens)	**294 bytes (∼73 tokens via CAS URI)**	**99.85% Token Reduction**

Table 8: **Experiment 5 (RQ5): Live Multi-File Corruption Recovery and `cargo check` Verification** (5 trials per scale, Mean ± StdDev).

Corrupted Rust Modules	Sequential File Restore (Mean ± SD)	Aegis Atomic CoW Rollback (Mean ± SD)	Kernel I/O Speedup	Post-Rollback `cargo check`
10 modules (640 KB)	0.76 ± 0.03 ms	$\mathbf{0.62 \pm 0.01}$ **ms**	**1.2×**	`PASSED (Exit 0)`
50 modules (3.2 MB)	3.99 ± 0.06 ms	$\mathbf{1.81 \pm 0.07}$ **ms**	**2.2×**	`PASSED (Exit 0)`
200 modules (12.8 MB)	15.93 ± 0.22 ms	$\mathbf{6.51 \pm 0.19}$ **ms**	**2.4×**	`PASSED (Exit 0)`

overlapping line conflicts, atomically aborts the conflicting second commit without corrupting the host file, and returns an exact baseline line-number conflict diagnostic so the agent can coordinate via `aos-ipc`.

- **CAS Context Token Savings:** Replacing inline 64 KB specification handoffs with `artifact://` CAS URIs reduces IPC message payload bytes across 3 handoffs from 196,608 bytes (∼49,152 tokens) to 294 bytes (∼73 tokens), achieving a **99.85% context token reduction** (Table 7).

5.5 Experiment 5 (RQ5): Time-Travel Rollback and `cargo check` Verification

Finally, we evaluate recovery from catastrophic agent hallucinations where an agent introduces syntax errors (`BROKEN_RUST_SYNTAX_ERROR !!!`) across 10, 50, and 200 Rust module files (`src/mod_*.rs`, 64 KB per module). As shown in Table 8, Aegis OS's atomic directory-tree `workspace_rollback` restores all 200 corrupted modules in 6.51 ± 0.19 **ms** (**2.4× faster** than sequential file-by-file I/O restoration and over **10,000× faster** than multi-turn LLM-driven file reverts). Immediately after rollback, our benchmark harness executes `cargo check -offline` inside the restored shadow workspace, confirming 100% compilation recovery across all scales.

6 Cross-Platform Mapping, Discussion, and Limitations

Cross-Platform Kernel Primitive Duality (macOS vs. Linux). While our live empirical evaluation in §5 was conducted on Apple Silicon (`macos-aarch64`), every Aegis OS subsystem is architected around platform-agnostic Rust traits (`TransactionalWorkspace`, `SandboxDriver`) that map cleanly onto native Linux kernel primitives, as detailed in Table 9. On Linux hosts formatted with Btrfs or XFS (`reflink=1`), `aos-vfs` uses extent-level `FICLONE` reflinks [13] (or `OverlayFS` upperdirs [17] on ext4) to achieve the same $O(1)$ CoW workspace provisioning. Similarly, `aos-sandbox` executes `/usr/bin/bwrap` on Linux to enforce unprivileged user, mount, PID, and network namespaces, while Tier 2 provides the control-plane interface for `libkrun`/Firecracker microVMs [1].

Filesystem Read Denylist vs. Allowlist Trade-offs on Developer Workstations. On Linux (`bwrap`), mounting an isolated root filesystem with selective read-only bind mounts (`-ro-bind /usr /usr`) naturally enforces a strict read allowlist. On macOS, however, XNU's dynamic linker (`dyld` shared cache), Xcode toolchains, and Homebrew libraries traverse numerous system paths under `/System`, `/Library`, and `/opt/homebrew`. Consequently, our macOS Seatbelt profile combines a *strict allowlist on file writes* (`(deny file-write*)`) with a *targeted regex denylist on file reads* covering high-value credential stores (`.ssh`, `.aws`, `.gnupg`, `.kube`, `.config/gh`, `.npmrc`, `.pypirc`, and `Library/Keychains`). Users requiring strict read isolation from sibling source repositories on macOS can additionally pass custom `(deny file-read* (subpath ...))` rules for their parent project directories while keeping toolchain paths readable.

HTTPS TLS Termination and Boundary Secret Injection. In our current implementation, `AegisEgressGateway` operates as an L7 HTTP boundary gateway and MCP `egress_http_request` sanitizer, pinning agent identities per listener socket (`start_local_proxy_with_bound_identity`) and enforcing kernel-level Seatbelt `(deny network-outbound)` so child processes cannot bypass the local proxy. When legacy CLI binaries inside `proc_exec` initiate direct TLS (`HTTPS CONNECT`) tunnels, inspecting headers requires the local proxy to terminate TLS using a session-scoped ephemeral Certificate Authority (CA) injected into the sandbox's `SSL_CERT_FILE` environment vari-

Table 9: **Cross-Platform Operating System Primitive Mapping** between macOS Developer Workstations and Linux Cloud Hosts in Aegis OS.

Aegis OS Kernel Abstraction	macOS Workstation Implementation	Linux Workstation / Container Host	Multi-Tenant Cloud Host (Tier 2)
Copy-on-Write Workspace (`aos-vfs`)	APFS `libc::clonefile(src, dst, 0)`	Btrfs/XFS `ioctl(FICLONE)` or `OverlayFS`	Virtio-fs / Block CoW Snapshot
Filesystem MAC (`aos-sandbox`)	XNU Seatbelt (`/usr/bin/sandbox-exec`)	Bubblewrap (`bwrap` Mount/User NS) + Landlock LSM	Hardware KVM (`libkrun` / Firecracker)
Credential Read Blocking (`~/.ssh`)	Seatbelt (`deny file-read* (regex ...)`)	Empty tmpfs bind-mount over restricted paths	Unmounted host volumes (Guest kernel)
Syscall / Process Filtering	Seatbelt SBPL (`deny file-write*`)	`seccomp-bpf` filter + PID/IPC Namespace	Guest microVM kernel isolation
Zero-Trust Egress (`aos-egress`)	Local TCP Proxy + Seatbelt `network-outbound`	Network Namespace (`CLONE_NEWNET`) + Proxy veth	Tap/vsock Egress Gateway

able. Extending `aos-egress` with automated per-sandbox ephemeral CA generation and full upstream TLS forwarding is a natural engineering extension.

Syntactic 3-Way Merging vs. Semantic Integration Conflicts. Our LCS Diff3 3-Way Merge Engine (`three_way_merge_lines`) [10] resolves 100% of disjoint line-level edits (including multi-line insertions) and catches 100% of overlapping line conflicts (Table 7). However, two agents could make syntactically disjoint edits that introduce a *semantic conflict* (e.g., Agent A renames a function signature at Line 10 while Agent B adds a call to the old function name at Line 120). Aegis OS mitigates semantic conflicts by allowing the committing agent (or a pre-commit verification hook) to execute `cargo test` inside the sandbox after merging peer changes from `host/` into `upper_workspace/` prior to finalizing `workspace_commit`, and by supporting instant `workspace_rollback` if post-merge compilation fails.

7 Related Work

LLM Agent Sandboxes and Prompt Injection Defenses. Benchmarks such as SWE-bench [9], SWE-agent [20], OpenHands [15], ToolEmu [14], AgentDojo [6], and InjecAgent [21] highlight both the capabilities and vulnerability of tool-using agents to indirect prompt injection [7]. Recent defenses such as SecGPT [19], CaMeL [5], and FIDES [3] propose information-flow control or dual-LLM privilege separation at the prompt/interpreter level. Aegis OS is complementary to model-level defenses: it enforces mandatory access control at the *operating system kernel and network boundary* regardless of how the agent planner is implemented. Unlike container-per-task sandboxes (e.g., Docker), which incur seconds of startup latency and isolate agents from the developer's live local repository, Aegis OS combines sub-6ms APFS CoW shadow workspaces [13, 17] with Seatbelt/Bubblewrap mandatory access control [11] and an LCS Diff3 3-Way Merge Engine [10] directly over the live repository.

Agent Operating Systems and Multi-Agent Frameworks. AIOS [12] integrates an LLM kernel scheduler above application agents, while AutoGen [18] and MetaGPT [8] orchestrate multi-agent conversations in Python. In contrast, Aegis OS adopts a strict "Environment Kernel Only" design aligned with classical capability systems such as Capsicum [16], Mbox [11], Cedar authorization [4], and Firecracker microVMs [1], serving any MCP-compatible client (`agy`, `claude`, `codex`) without dictating agent reasoning logic.

8 Conclusion

We presented **Aegis OS**, a transactional, multi-tenant Agent Operating System Runtime that resolves the Security–Autonomy Dilemma for autonomous coding agents. By combining APFS Copy-on-Write shadow workspaces, LCS Diff3 3-Way multi-agent merge commits, kernel-enforced Seatbelt/Bubblewrap sandboxes with outbound network confinement, Zero-Trust boundary secret injection, and first-class IPC + CAS primitives, Aegis OS eliminates 94.19%–99.88% of human confirmation prompts while reducing adversarial attack success from 100.0% to 0.0%.

References

[1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. Firecracker: Lightweight virtualization for serverless applications. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 419–434. USENIX Association, 2020.

[2] Anthropic. Model context protocol (MCP): An open protocol for connecting AI applications to external systems. https://modelcontextprotocol.io, 2024.

[3] Manuel Costa, Boris Köpf, Aecio Ribeiro, Andrew Roscoe, Mark Russinovich, and M. Schut. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643*, 2025.

[4] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew Wells. Cedar: A new language for expres-

sive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):670–697, 2024.

[5] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Soares, Nicholas Cano, Mathias Payer, and Florian Tramèr. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.

[6] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, volume 37, 2024.

[7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.

[8] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[10] Sanjeev Khanna, Keshav Kunal, and Benjamin C. Pierce. A formal investigation of Diff3. In *27th International Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS)*, pages 485–496. Springer, 2007.

[11] Taesoo Kim and Nickolai Zeldovich. Practical sandboxing for non-root users. In *2013 USENIX Annual Technical Conference (USENIX ATC 13)*, pages 139–144. USENIX Association, 2013.

[12] Kai Mei, Xi Zhu, Wujiang Xu, Wenyue Hua, Mingyu Jin, Zelong Li, Shuyuan Xu, Ruosong Ye, Yingqiang Ge, and Yongfeng Zhang. AIOS: LLM agent operating system. In *Conference on Language Modeling (COLM)*, 2025. arXiv:2403.16971.

[13] Ohad Rodeh, Josef Bacik, and Chris Mason. BTRFS: The Linux b-tree filesystem. *ACM Transactions on Storage (TOS)*, 9(3):9:1–9:32, 2013.

[14] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an LM-emulated sandbox. In *Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[15] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[16] Robert N. M. Watson, Jonathan Anderson, Ben Laurie, and Kris Kennaway. Capsicum: Practical capabilities for UNIX. In *19th USENIX Security Symposium (USENIX Security 10)*, pages 29–46. USENIX Association, 2010.

[17] Charles P. Wright, Jay Dave, Puja Gupta, Harikesavan Krishnan, David P. Quigley, Erez Zadok, and Mohammad Nayyer Zubair. Versatility and Unix semantics in a fan-out unification file system. In *ACM Transactions on Storage (TOS)*, volume 2, pages 74–105, 2006.

[18] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *First Conference on Language Modeling (COLM)*, 2024.

[19] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. SecGPT: An execution isolation architecture for large language model based systems. *arXiv preprint arXiv:2403.04960*, 2024.

[20] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.

[21] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt

injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 10471–10506. Association for Computational Linguistics, 2024.

A Appendix: Reproducibility and Artifact Guide

The complete open-source implementation of Aegis OS and the reproducible benchmark suite are publicly available at https://github.com/heurion-org/aegis-os. All source crates (`aos-core`, `aos-policy`, `aos-vfs`, `aos-ipc`, `aos-sandbox`, `aos-egress`, `aos-recorder`, `aos-mcp-gateway`, and `aos-cli`) and the live empirical benchmark suite are self-contained in the `aegis-os` workspace. To reproduce all five experiments and regenerate `BENCHMARK_REPORT.md` and `BENCHMARK_REPORT.json`:

```
cargo test --workspace
cargo run --release --bin aegis-os -- bench \
  --report-out BENCHMARK_REPORT.md
```